Stride

The Unified Student Academic and Campus Application

Project Review 2

System Design, UML/ERD Models and Dashboard Development

Department of Computing and Information Systems
Botswana Accountancy College

BSc (Hons) Computer Systems Engineering
With Resego Rabalone

Nyasha Scott Nyandoro (CSE24-146)
Ewetse Ndala (CSE24-020)
Atang Zibo Rari (CSE24-090)
Bame Kaosi Mosimakoko (CSE24-076)
Katso Kgengwenyane (CSE24-083)

Submission date: 9 October 2026
(Deadline assumed to be the end of 9 October 2026, as instructed.)

Purpose and scope

This report presents the system design deliverables for Project Review 2: the system architecture, use-case model, entity relationship diagram (ERD), UML class diagram, attendance sequence model and dashboard design. Stride is proposed as a JavaFX desktop application for Botswana Accountancy College (BAC), backed by JDBC and MySQL 8. It provides a unified entry point to common student and campus tasks while linking to existing college platforms rather than replacing them.

The three user roles are **Student**, **Lecturer** and **Administrator**. Student functions include viewing a timetable and attendance, checking in to class, finding free rooms and building a study schedule. Lecturers open attendance sessions, correct attendance and post announcements. Administrators manage users, courses/modules and timetables, and access reports. Existing platforms are accessed through quick links; live integration with BAC systems is outside the current prototype scope.

1 System architecture

The architecture separates the interface, business rules, data access and persistence. JavaFX screens call Java service classes; services apply validation and scheduling rules before using JDBC data-access classes to query or update MySQL. This separation reduces direct database logic in the interface and makes the business rules easier to test. External college platforms are represented as quick links, not as systems controlled by Stride.

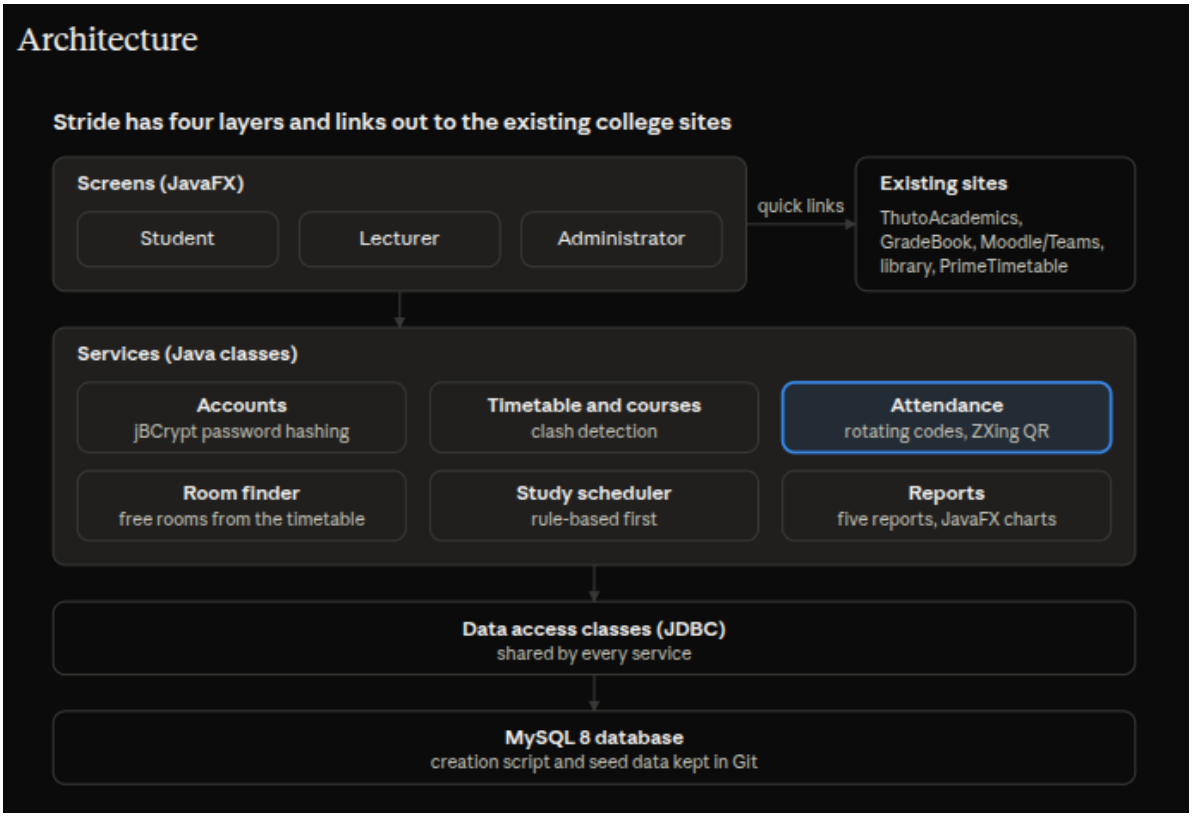


Figure 1: Stride four-layer architecture and links to existing college sites.

2 Use-case model and functional scope

The use-case model maps each role to its main system interactions. Authentication is the entry point for role-appropriate functionality. Student use cases focus on self-service and campus navigation; lecturer use cases focus on teaching and attendance; administrator use cases focus on configuration and reporting.

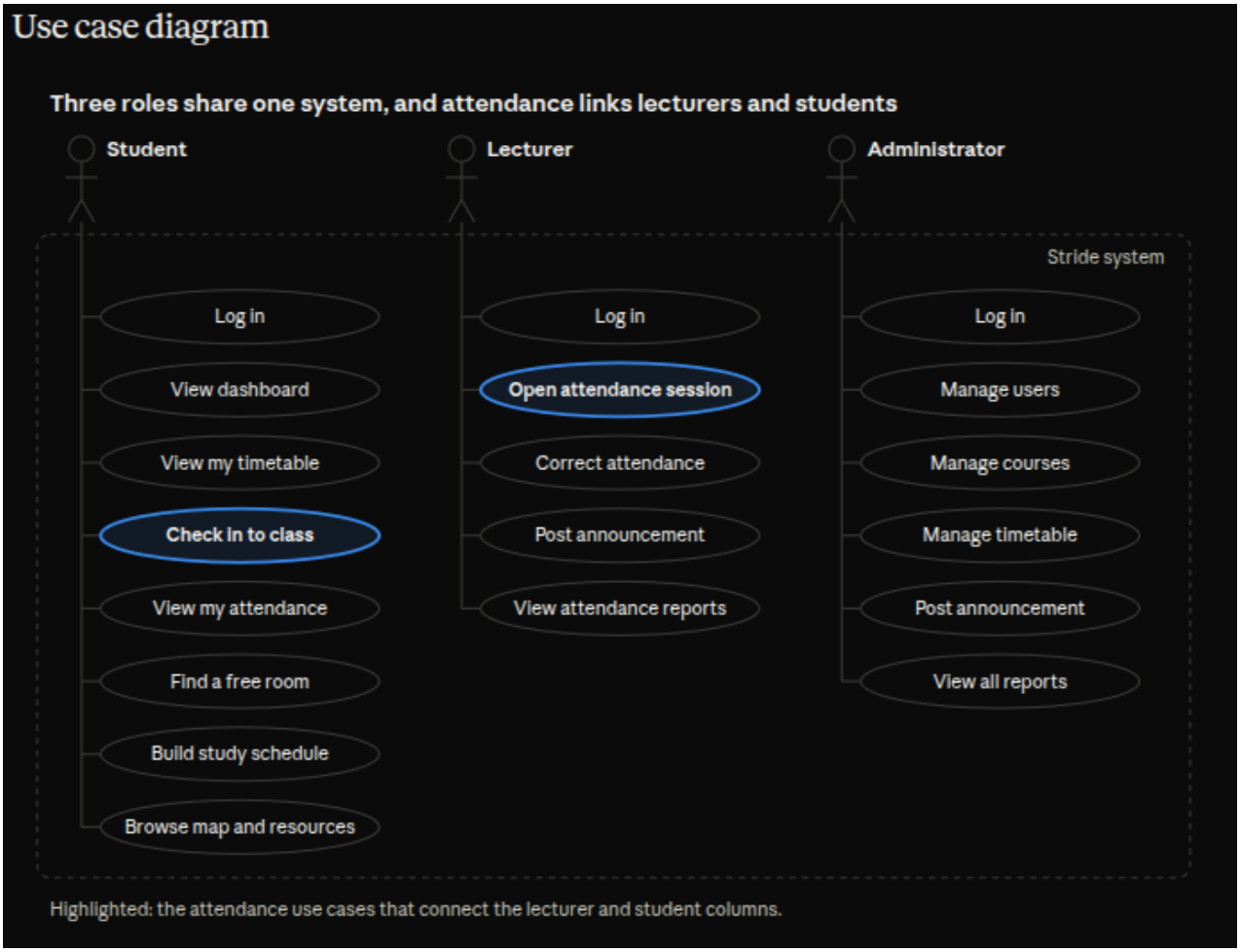


Figure 2: Use-case diagram showing Student, Lecturer and Administrator responsibilities.

3 Entity relationship diagram (ERD)

The ERD models the core records required for identity, teaching schedules, campus resources and attendance. `users` stores account and role information; `modules` identifies courses/modules and lecturers; `enrolments` links students to modules; `rooms` and `timetable_entries` represent scheduled teaching and room allocation. `attendance_sessions` stores the session code and validity window, while `check_ins` records a student's attendance result. Announcements and resources support dashboard content.

The main integrity rule is that a check-in must refer to an attendance session and a student enrolled in the relevant module. Session expiry, session status and duplicate submissions must be checked by the service layer; database constraints should also prevent duplicate check-ins for the same student and session. The diagram is the proposed logical model and should be checked against the final SQL schema before implementation is described as validated.

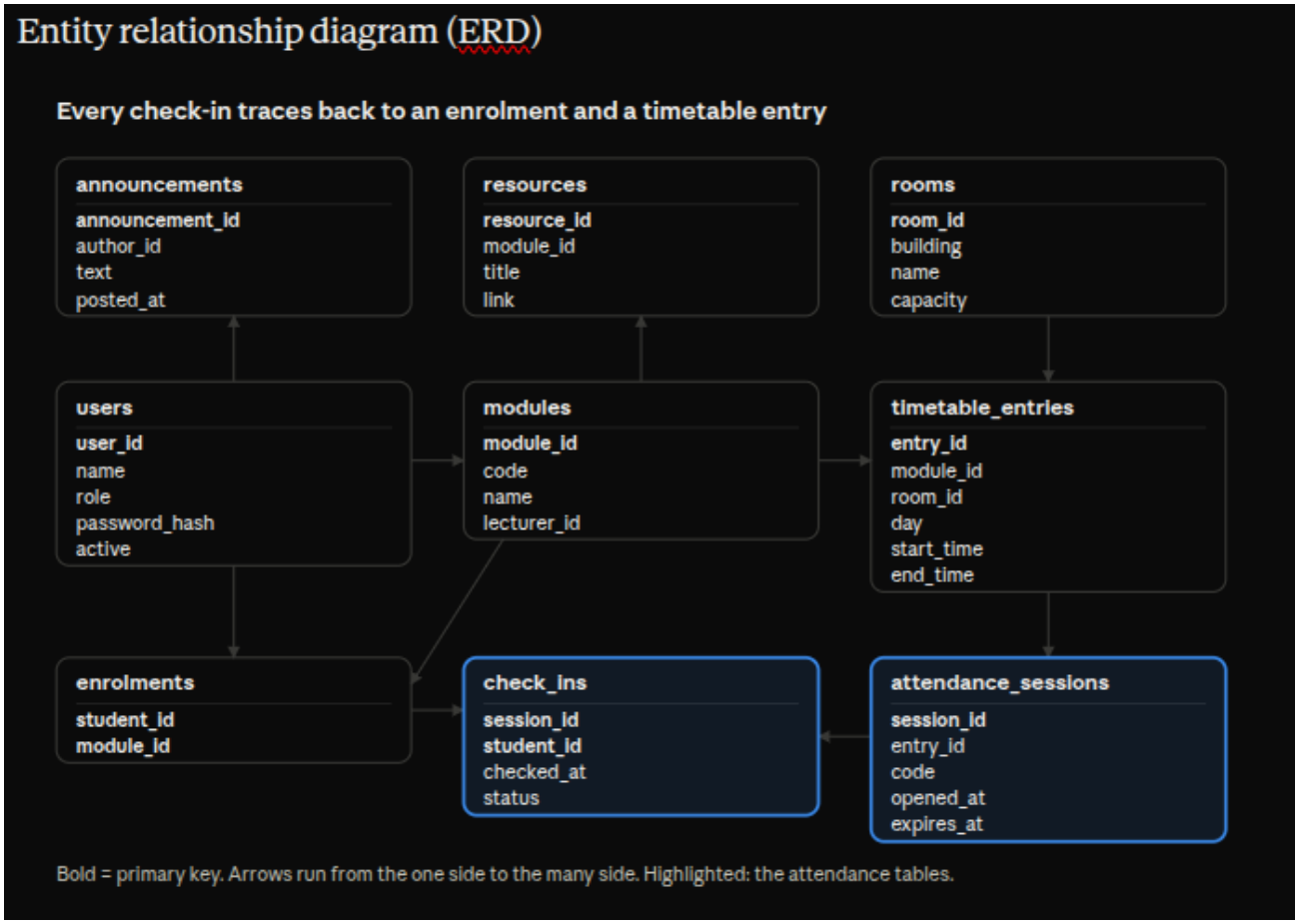


Figure 3: Proposed ERD showing core entities, keys and relationships.

4 UML class diagram

The class model translates the main domain concepts into object-oriented responsibilities. `User` is the common account concept, with Student, Lecturer and Administrator specialisations. Module, TimetableEntry and Room model teaching and location data. AttendanceSession represents an active, time-limited class session; CheckIn represents an individual student's attendance event. The operations shown indicate responsibilities such as opening a session, checking a code, validating a check-in, finding a free room and managing reports.

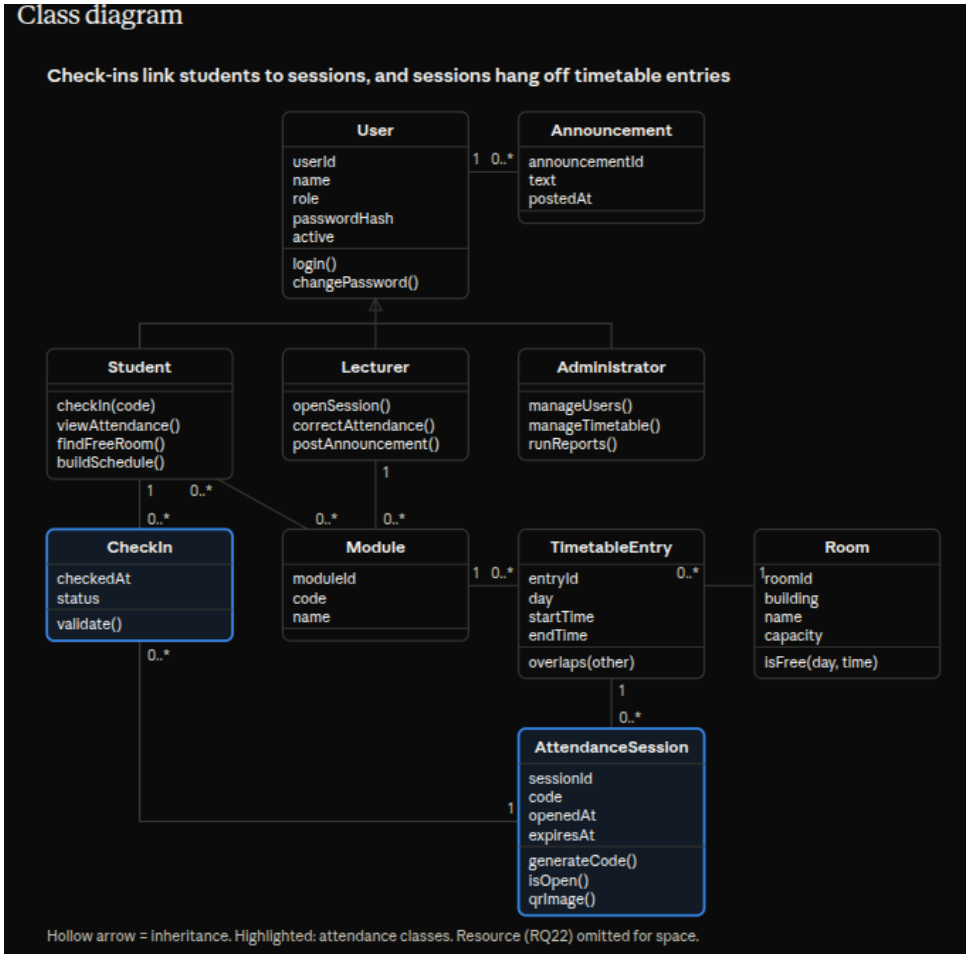


Figure 4: Domain UML class diagram, including inheritance, operations and multiplicities.

5 Attendance check-in sequence

The sequence model details the key interaction between lecturer, student, Stride and MySQL. A lecturer opens a session; Stride stores the session code and expiry; the student enters the code; and the system validates the code, session and enrolment before saving a check-in. The user receives a confirmation or a rejection reason. The system must not create a valid attendance record when validation fails.

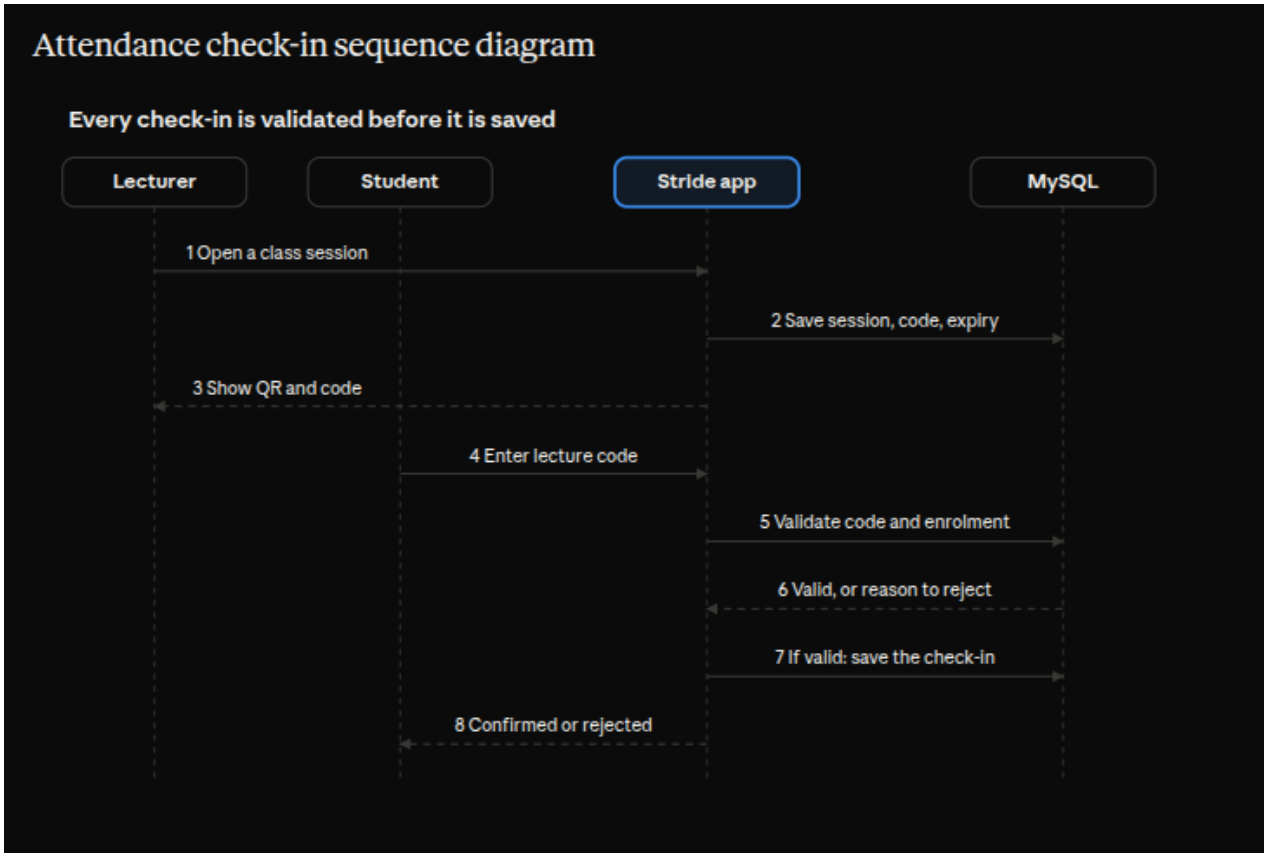


Figure 5: Sequence diagram for attendance session creation and student check-in.

6 Dashboard design and implementation approach

The student dashboard prioritises today's classes and check-in actions, then shows current free rooms, announcements and quick links. Navigation provides access to timetable, attendance, rooms and map, study planner and resources. The lecturer dashboard should emphasise session controls and attendance reports, while the administrator dashboard should emphasise user, course and timetable management. Role-specific views reduce clutter and keep common tasks easy to locate.

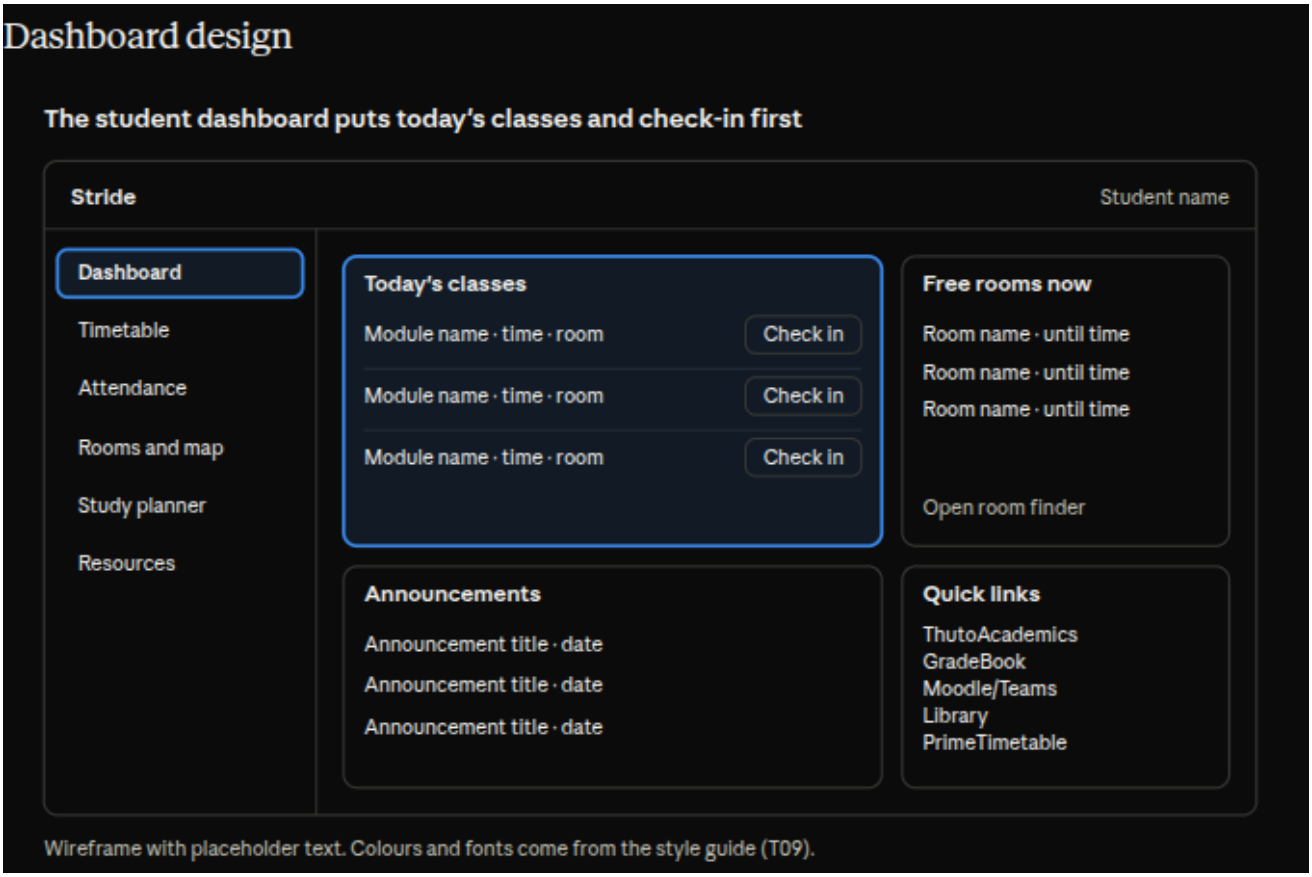


Figure 6: Student dashboard wireframe with sample placeholder data.

The supplied dashboard is a design prototype, not evidence of a fully connected JavaFX implementation. During development, each panel should be connected through service and JDBC classes, with loading, empty and error states. Only synthetic/sample data should be used in demonstrations until access and data-handling approvals are in place.

7 Design rules and validation priorities

- **Access control:** authenticate users and restrict operations by role; never rely on hiding a button alone as authorisation.
- **Attendance integrity:** reject expired or closed sessions, incorrect codes, students not enrolled in the module, and duplicate check-ins.
- **Timetable and room logic:** detect overlapping time intervals; a room is free only when no timetable entry overlaps the requested interval.
- **Database integrity:** define primary and foreign keys, appropriate uniqueness constraints and required fields in the MySQL schema.
- **Model consistency:** align ERD entities and class names with the executed SQL schema and final Java implementation.

The immediate validation tasks are to execute the schema in MySQL 8, verify the relationships and constraints, test the attendance success and rejection paths, test room availability against timetable overlaps, and review each dashboard action for the correct role. These are proposed validation activities; no test results are claimed in this design report.

Conclusion

The system models describe how Stride's roles, JavaFX screens, service layer, JDBC data access and MySQL records fit together. The ERD establishes the data relationships, the class diagram assigns responsibilities to domain objects, the sequence diagram specifies the attendance flow, and the dashboard wireframe communicates the intended student-facing layout. The next implementation step is to align the diagrams with the executable schema and Java classes, then verify the stated rules through documented tests.